



UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Course Name: Operating System Lab

Course Code: CSE 402

LAB REPORT

Lab Report Number : 03

Lab Report Name : Round Robin (RR) Scheduling Algorithm

Date of Submission : 17.08.2026

Submitted By:

Name : Ejaj Ahmmod Lemon

ID : 23101160

Section: C2

Submitted To:

Atia Rahman Orthi

Lecturer

University of Asia Pacific

1. Problem Statement:

An application is to be developed to implement the **Round Robin (RR) CPU Scheduling Algorithm**, which is a preemptive scheduling technique designed to provide fair CPU allocation among multiple processes. In Round Robin scheduling, each process is assigned a fixed amount of CPU time known as the **Time Quantum**. A process is allowed to execute for at most one time quantum. If the process does not complete within this time, it is preempted and placed at the end of the ready queue to wait for its next turn.

For this lab experiment, the scheduling algorithm is implemented using **6 processes** with a **time quantum of 2 units**. Each process is defined by its **Process ID (PID)**, **Arrival Time (AT)**, and **Burst Time (BT)**. The processes are scheduled according to their arrival times and are executed cyclically until all processes are completed.

The sample input is given below:

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	7
P2	1	4
P3	2	15
P4	3	11
P5	4	20
P6	4	9

Number of Processes: 6

Time Quantum: 2 units

The application should generate the scheduling sequence and calculate the required performance measures, including **Completion Time (CT)**, **Turnaround Time (TAT)**, and **Waiting Time (WT)** for each process. The implementation should also correctly handle processes arriving at different times and maintain the ready queue according to the Round Robin scheduling policy.

2. Objectives:

The objectives of this lab experiment are:

- To implement the **Round Robin (RR) CPU scheduling algorithm** using a fixed time quantum.
- To understand the effect of different **time quantum values** on process execution and overall scheduling performance.
- To calculate and analyze important scheduling performance metrics, including **Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT)**.
- To understand how the Round Robin algorithm provides **fair and equitable CPU time** to multiple processes.
- To gain practical experience with a **preemptive CPU scheduling technique** and understand its execution mechanism.

3. Source Code Screenshot:

```
from collections import deque
processes = [
    ["P1", 0, 7],
    ["P2", 1, 4],
    ["P3", 2, 15],
    ["P4", 3, 11],
    ["P5", 4, 20],
    ["P6", 4, 9]
]
time_quantum = 5
n = len(processes)
remaining = [p[2] for p in processes]
ct = [0] * n
queue = deque()
time = 0
completed = 0
visited = [False] * n
gnatt = []
while completed < n:
    for i in range(n):
        if processes[i][1] <= time and not visited[i]:
            queue.append(i)
            visited[i] = True
    if not queue:
        next_time = min(
            processes[i][1]
            for i in range(n)
            if not visited[i]
        )
        time = next_time
        continue
    i = queue.popleft()
    start_time = time
    execution_time = min(time_quantum, remaining[i])
```

```

40         queue.append(j)
41         visited[j] = True
42         if remaining[i] > 0:
43             queue.append(i)
44         else:
45             ct[i] = time
46             completed += 1
47     tat = [0] * n
48     wt = [0] * n
49
50     for i in range(n):
51         tat[i] = ct[i] - processes[i][1]
52         wt[i] = tat[i] - processes[i][2]
53     print("\nProcess\tAT\tBT\tCT\tTAT\tWT")
54     for i in range(n):
55         print(
56             f"{processes[i][0]}\t"
57             f"{processes[i][1]}\t"
58             f"{processes[i][2]}\t"
59             f"{ct[i]}\t"
60             f"{tat[i]}\t"
61             f"{wt[i]}"
62         )
63     avg_tat = sum(tat) / n
64     avg_wt = sum(wt) / n
65     print("\nAverage Turnaround Time:", avg_tat)
66     print("Average Waiting Time:", avg_wt)
67     print("\nGantt Chart:")
68     for process, start, end in gnatt:
69         print(f"| {process} ", end="")
70     print("|")
71     for process, start, end in gnatt:
72         print(f"{start:<4}", end="")
73     print(time)

```

Figure 1: Source Code of the **Round Robin** Algorithm

```

from collections import deque
processes = [
    ["P1", 0, 7],
    ["P2", 1, 4],
    ["P3", 2, 15],
    ["P4", 3, 11],
    ["P5", 4, 20],
    ["P6", 4, 9]
]
time_quantum = 5
def fcfs(processes):
    time = 0
    result = []
    for pid, at, bt in processes:
        if time < at:
            time = at
        ct = time + bt
        tat = ct - at
        wt = tat - bt
        result.append([pid, at, bt, ct, tat, wt])
        time = ct
    return result

def sjf(processes):
    remaining = processes.copy()
    result = []
    time = 0
    while remaining:
        available = [
            p for p in remaining
            if p[1] <= time
        ]
        if not available:
            time = min(p[1] for p in remaining)

```

```

49 def round_robin(processes, quantum):
50
51     n = len(processes)
52
53     remaining = [p[2] for p in processes]
54     ct = [0] * n
55
56     queue = deque()
57
58     visited = [False] * n
59
60     time = 0
61     completed = 0
62
63     while completed < n:
64         for i in range(n):
65             if processes[i][1] <= time and not visited[i]:
66                 queue.append(i)
67                 visited[i] = True
68
69             if not queue:
70                 time = min(
71                     processes[i][1]
72                     for i in range(n)
73                     if not visited[i]
74                 )
75                 continue
76
77             i = queue.popleft()
78
79             execution_time = min(
80                 quantum,

```

```

def Round_Robin(processes, quantum):
    result.append([
        pid, at, bt, ct[i], tat, wt
    ])

    return result

def print_result(name, result):
    print("\n" + "=" * 55)
    print(name)
    print("=" * 55)

    print("Process\tAT\tBT\tCT\tTAT\tWT")

    for r in result:
        print(
            f"{r[0]}\t"
            f"{r[1]}\t"
            f"{r[2]}\t"
            f"{r[3]}\t"
            f"{r[4]}\t"
            f"{r[5]}"
        )

    avg_tat = sum(r[4] for r in result) / len(result)
    avg_wt = sum(r[5] for r in result) / len(result)

    print("\nAverage Turnaround Time =", round(avg_tat, 2))
    print("Average Waiting Time      =", round(avg_wt, 2))

    return avg_tat, avg_wt

```

Figure 2: Source Code of the Comparison of FCFS vs SJF vs **Round Robin** Algorithm

4. Output Screenshot:

```

Process AT      BT      CT      TAT      WT
P1      0       7      31      31      24
P2      1       4       9       8       4
P3      2      15     55     53     38
P4      3      11     56     53     42
P5      4      20     66     62     42
P6      4       9     50     46     37

Average Turnaround Time: 42.166666666666664
Average Waiting Time: 31.166666666666668

Gantt Chart:
| P1 | P2 | P3 | P4 | P5 | P6 | P1 | P3 | P4 | P5 | P6 | P3 | P4 | P5 | P5 |
0   5   9  14  19  24  29  31  36  41  46  50  55  56  61  66

...Program finished with exit code 0
Press ENTER to exit console.

```

Figure 3: Execution Output of the **Round Robin** Algorithm

FCFS					
Process	AT	BT	CT	TAT	WT
P1	0	7	7	7	0
P2	1	4	11	10	6
P3	2	15	26	24	9
P4	3	11	37	34	23
P5	4	20	57	53	33
P6	4	9	66	62	53
Average Turnaround Time = 31.67					
Average Waiting Time = 20.67					
SJF					
Process	AT	BT	CT	TAT	WT
P1	0	7	7	7	0
P2	1	4	11	10	6
P6	4	9	20	16	7
P4	3	11	31	28	17
P3	2	15	46	44	29
P5	4	20	66	62	42
Average Turnaround Time = 27.83					
Average Waiting Time = 16.83					
ROUND ROBIN (Time Quantum = 5)					
Process	AT	BT	CT	TAT	WT
P1	0	7	31	31	24
P2	1	4	9	8	4
P3	2	15	55	53	38
P4	3	11	56	53	42
P5	4	20	66	62	42
P6	4	9	50	46	37
Average Turnaround Time = 42.17					
Average Waiting Time = 31.17					
PERFORMANCE COMPARISON					
Algorithm	Average WT	Average TAT			
FCFS	20.67	31.67			
SJF	16.83	27.83			
Round Robin	31.17	42.17			
BEST PERFORMANCE					
Best Algorithm = SJF					
Lowest Average Waiting Time = 16.83					

Figure 4: Execution Output of the Comparison of FCFS vs SJF vs **Round Robin** Algorithm

The program was executed with 6 processes and a time quantum of 2 time units. The results demonstrate how the Round Robin scheduling algorithm distributes CPU time among the processes in a cyclic manner:

- Process P1 arrived at time 0, required 7 time units, and finished at time 29.
- Process P2 arrived at time 1, required 4 time units, and finished at time 16.
- Process P3 arrived at time 2, required 15 time units, and finished at time 60.
- Process P4 arrived at time 3, required 11 time units, and finished at time 53.
- Process P5 arrived at time 4, required 20 time units, and finished at time 66.
- Process P6 arrived at time 4, required 9 time units, and finished at time 50.

The average turnaround time was 42.17 time units, while the average waiting time was 31.17 time units. These results demonstrate how the Round Robin algorithm repeatedly allocates the CPU to each ready process for a maximum of two time units before moving to the next process.

5. Discussion:

Discussion and Learning

Working with the **Round Robin CPU scheduling algorithm** provided a clear understanding of how a preemptive scheduling technique can distribute CPU time fairly among multiple processes. The main advantage of Round Robin is that each process receives a fixed amount of CPU time before the scheduler moves to the next process. This prevents any single process from occupying the CPU for an excessive amount of time.

The **time quantum of 2 units** used in this experiment had a significant effect on the scheduling behavior:

- If the time quantum is **too small**, the number of context switches increases, which can introduce additional overhead.
- If the time quantum is **too large**, the algorithm gradually behaves more like **First-Come, First-Served (FCFS)** scheduling.
- An appropriate time quantum provides a good balance between **responsiveness, fairness, and scheduling overhead**.

The implementation used a **queue** to maintain the order of processes waiting for CPU execution. Processes were added to the ready queue according to their arrival times, and after using their assigned time quantum, an incomplete process was placed at the end of the queue for its next

turn. A **visited array** was also used to ensure that processes were not unnecessarily added to the queue multiple times.

One important observation from this experiment is that Round Robin provides good **responsiveness and fairness**, particularly in systems where multiple processes need to share the CPU. Shorter processes can often complete without waiting for longer processes to finish completely, while longer processes also continue to receive regular opportunities for execution.

Overall, this experiment demonstrated how Round Robin balances **fair CPU allocation, responsiveness, and context-switching overhead**, making it particularly suitable for **time-sharing and interactive systems**.

6. Conclusion:

Analysis

The performance results show a clear difference among the three CPU scheduling algorithms: **FCFS, SJF, and Round Robin**. For the given set of six processes, Round Robin was executed with a **time quantum of 5 units**.

The Round Robin results show an **average turnaround time of 42.17 time units** and an **average waiting time of 31.17 time units**. The relatively high waiting and turnaround times indicate that processes had to wait for multiple CPU turns because each process received only a limited amount of CPU time in each cycle.

The performance comparison obtained from the implementation is:

Algorithm	Average Waiting Time	Average Turnaround Time
FCFS	20.67	31.67
SJF	16.83	27.83
Round Robin	31.17	42.17

From the results, **SJF achieved the best performance**, with the lowest average waiting time of **16.83 time units** and the lowest average turnaround time of **27.83 time units**. Compared with FCFS, SJF reduced the average waiting time by **3.84 time units** and the average turnaround time by **3.84 time units**.

FCFS performed better than Round Robin in this particular test case, with an average waiting time of **20.67** compared with **31.17** for Round Robin. Similarly, the average turnaround time of FCFS (**31.67**) was lower than that of Round Robin (**42.17**).

The Round Robin result is influenced by the selected **time quantum of 5 units**. Although Round Robin provides fair CPU access and prevents a single process from continuously occupying the CPU, repeated preemption and waiting for subsequent turns can increase the waiting and turnaround times for this particular workload.

Therefore, based on the measured results, **SJF is the most efficient algorithm for this specific workload when average waiting time and turnaround time are the primary performance criteria**. However, the result does not mean SJF is always the best choice, because each scheduling algorithm is designed for different system requirements.

Conclusion and Comparison of CPU Scheduling Algorithms

The experiment demonstrates the different characteristics and performance of **FCFS, SJF-NP, and Round Robin** CPU scheduling algorithms. Each algorithm follows a different scheduling strategy and therefore produces different waiting and turnaround times.

FCFS (First-Come, First-Served) is the simplest scheduling algorithm. It executes processes according to their arrival order and does not require prior knowledge of burst times. In this experiment, FCFS achieved an average waiting time of **20.67 time units** and an average turnaround time of **31.67 time units**. Its main advantages are simplicity, fairness based on arrival order, and low scheduling overhead. However, it may suffer from the **convoy effect**, where a long process delays several shorter processes.

SJF-NP (Shortest Job First–Non-Preemptive) achieved the best performance in this experiment. It produced the lowest average waiting time of **16.83 time units** and the lowest average turnaround time of **27.83 time units**. This demonstrates the main advantage of SJF: selecting shorter jobs first can significantly reduce the average waiting time. However, SJF requires knowledge or estimation of burst times and may cause **starvation** for longer processes if shorter processes continuously arrive.

Round Robin (RR) is a preemptive scheduling algorithm that provides each process with a fixed CPU time called the **time quantum**. With a time quantum of **5 units**, the experiment produced an average waiting time of **31.17 time units** and an average turnaround time of **42.17 time units**. Although Round Robin did not achieve the lowest performance metrics in this experiment, it provides better **fairness and responsiveness** because every ready process receives regular opportunities to execute. It is therefore particularly suitable for **time-sharing and interactive systems**.

Overall, the experimental results can be summarized as follows:

- **SJF** → Best performance in this experiment for minimizing average waiting and turnaround time.

- **FCFS** → Simple and easy to implement, but may suffer from the convoy effect.
- **Round Robin** → Provides fair CPU sharing and good responsiveness, but its performance depends heavily on the selected time quantum.

Therefore, the **best scheduling algorithm depends on the system requirements**. If minimizing average waiting time is the primary goal and burst times are known, **SJF** is highly effective. If simplicity and straightforward implementation are required, **FCFS** is a suitable choice. On the other hand, if fairness and responsiveness are more important, particularly in interactive systems, **Round Robin** is more appropriate.

For the specific workload used in this experiment, **SJF was the best-performing algorithm**, as confirmed by the program's performance comparison, with the lowest average waiting time of **16.83 time units**.